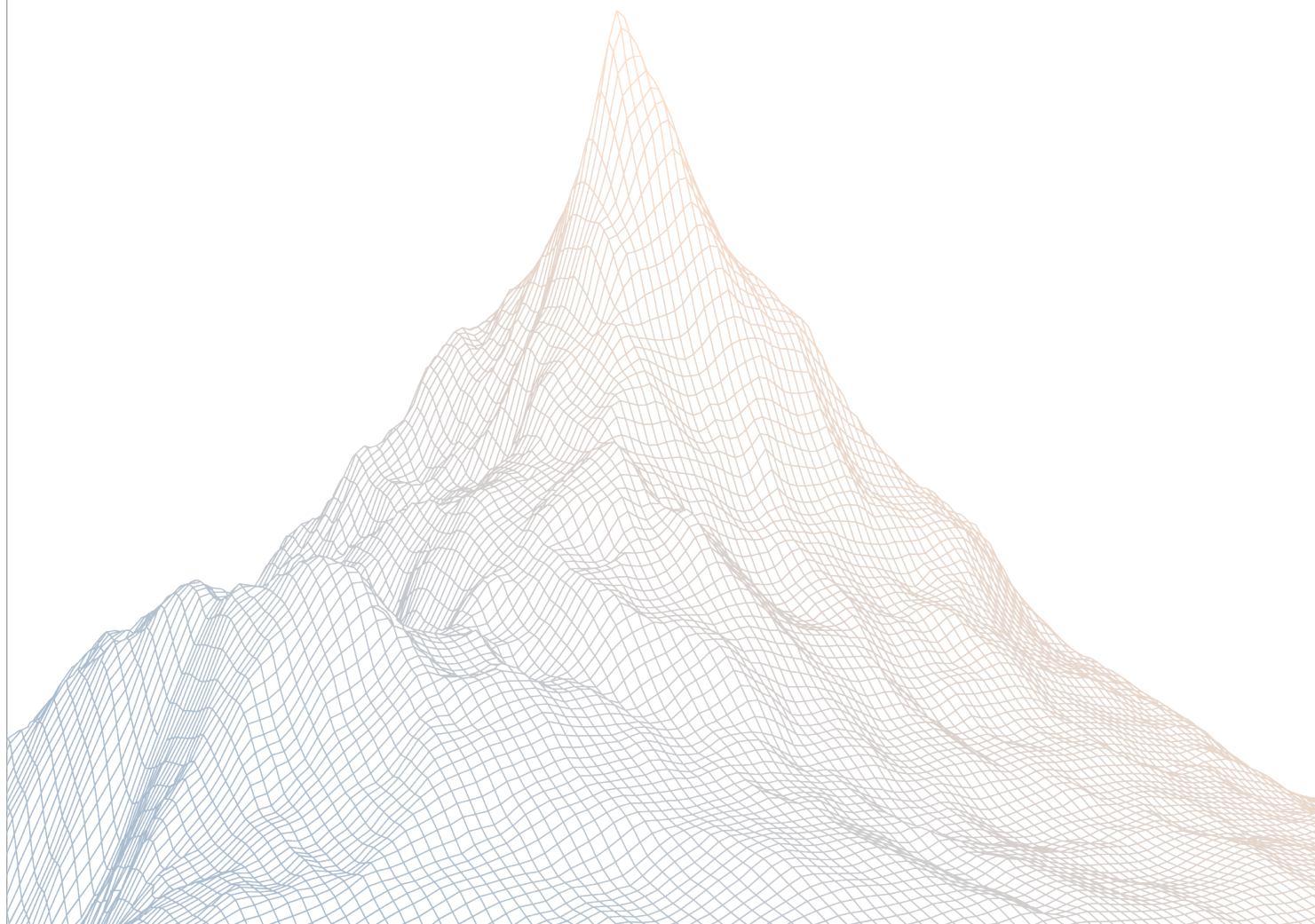


# Injective

## Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

May 20th to May 22th, 2026

AUDITED BY:

ether\_sky  
said

## Contents

|          |                          |          |
|----------|--------------------------|----------|
| <b>1</b> | <b>Introduction</b>      | <b>2</b> |
| 1.1      | About Zenith             | 3        |
| 1.2      | Disclaimer               | 3        |
| 1.3      | Risk Classification      | 3        |
| <hr/>    |                          |          |
| <b>2</b> | <b>Executive Summary</b> | <b>3</b> |
| 2.1      | About Injective          | 4        |
| 2.2      | Scope                    | 4        |
| 2.3      | Audit Timeline           | 6        |
| 2.4      | Issues Found             | 6        |
| <hr/>    |                          |          |
| <b>3</b> | <b>Findings Summary</b>  | <b>6</b> |
| <hr/>    |                          |          |
| <b>4</b> | <b>Findings</b>          | <b>7</b> |
| 4.1      | Medium Risk              | 8        |
| 4.2      | Low Risk                 | 13       |
| 4.3      | Informational            | 15       |

# 1

## Introduction

### 1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

---

### 1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

---

### 1.3 Risk Classification

| SEVERITY LEVEL     | IMPACT: HIGH | IMPACT: MEDIUM | IMPACT: LOW |
|--------------------|--------------|----------------|-------------|
| Likelihood: High   | Critical     | High           | Medium      |
| Likelihood: Medium | High         | Medium         | Low         |
| Likelihood: Low    | Medium       | Low            | Low         |

## 2

### Executive Summary

## 2.1 About Injective

Injective is a lightning fast interoperable layer one blockchain optimized for building premier Web3 finance applications. Injective provides developers with powerful plug-and-play modules for creating unmatched dApps. INJ is the native asset that powers Injective and its rapidly growing ecosystem. Injective is incubated by Binance and is backed by prominent investors such as Jump Crypto, Pantera and Mark Cuban.## About Injective

Injective is a lightning fast interoperable layer one blockchain optimized for building premier Web3 finance applications. Injective provides developers with powerful plug-and-play modules for creating unmatched dApps. INJ is the native asset that powers Injective and its rapidly growing ecosystem. Injective is incubated by Binance and is backed by prominent investors such as Jump Crypto, Pantera and Mark Cuban.

## 2.2 Scope

The engagement involved a review of the following targets:

|               |     |
|---------------|-----|
| <b>Target</b> | rfq |
|---------------|-----|

|                   |                                                                                                         |
|-------------------|---------------------------------------------------------------------------------------------------------|
| <b>Repository</b> | <a href="https://github.com/InjectiveLabs/rfq/pull/50">https://github.com/InjectiveLabs/rfq/pull/50</a> |
|-------------------|---------------------------------------------------------------------------------------------------------|

|                    |                                          |
|--------------------|------------------------------------------|
| <b>Commit Hash</b> | 38bcb918f0b2eddcea9933db21ad7ee7ad3e6965 |
|--------------------|------------------------------------------|

|              |                                                                                               |
|--------------|-----------------------------------------------------------------------------------------------|
| <b>Files</b> | Diff from b9c7434dda9374741e2f9ba91dc268ab0e3ed5d7<br>rfq-contract/contracts/rfq/src/**/*.*rs |
|--------------|-----------------------------------------------------------------------------------------------|

|                    |                                                                                                         |
|--------------------|---------------------------------------------------------------------------------------------------------|
| <b>Target</b>      | Mitigation Review                                                                                       |
| <b>Repository</b>  | <a href="https://github.com/InjectiveLabs/rfq/pull/50">https://github.com/InjectiveLabs/rfq/pull/50</a> |
| <b>Commit Hash</b> | 1c2bba5b67863495aa47f2914fa6e683778e8f1f                                                                |
| <b>Files</b>       | Diff from b9c7434dda9374741e2f9ba91dc268ab0e3ed5d7<br>rfq-contract/contracts/rfq/src/**/*.rs            |

## 2.3 Audit Timeline

|                     |                  |
|---------------------|------------------|
| <b>May 20, 2026</b> | Audit start      |
| <b>May 22, 2026</b> | Audit end        |
| <b>May 29, 2026</b> | Report published |

## 2.4 Issues Found

| SEVERITY            | COUNT     |
|---------------------|-----------|
| Critical Risk       | 0         |
| High Risk           | 0         |
| Medium Risk         | 3         |
| Low Risk            | 1         |
| Informational       | 7         |
| <b>Total Issues</b> | <b>11</b> |

# 3

## Findings Summary

| ID  | Description                                                                                                   | Status       |
|-----|---------------------------------------------------------------------------------------------------------------|--------------|
| M-1 | RFQ synthetic fills are not screened against Core min_notional before settlement                              | Resolved     |
| M-2 | The evm_chain_id should be empty for V1 signatures                                                            | Resolved     |
| M-3 | Long-lived TP/SL signed intents are not bound to the protected position lifecycle                             | Acknowledged |
| L-1 | Updating allowed_evm_chain_ids retroactively changes executability of already-signed V2 messages              | Acknowledged |
| I-1 | There is no need to validate the chain ID and contract address in V2 mode                                     | Acknowledged |
| I-2 | Taker-specific maker quotes dual-signed as V1 and V2 can bypass duplicate detection and fill twice            | Acknowledged |
| I-3 | The mode can not be Auto in the ensure_allowed_relayer function                                               | Resolved     |
| I-4 | There is no need to call the validate_signed_intent_shape function in Auto mode                               | Acknowledged |
| I-5 | V1 sign_mode bypasses the EVM-chain allowlist, so the allowlist is not a global signature policy              | Acknowledged |
| I-6 | Long TP/SL lifetime is granted solely by trigger type, allowing always-satisfied long-lived immediate intents | Acknowledged |
| I-7 | Signed taker_nonce_time_window_ms can remove the practical RFQ-id freshness bound                             | Resolved     |

# 4

## Findings

### 4.1 Medium Risk

A total of 3 medium risk findings were identified.

[M-1] RFQ synthetic fills are not screened against Core `min_notional` before settlement

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

#### Target

- [common.rs#L78-L83](#)
- [quote\\_validation.rs#L124-L240](#)
- [accept.rs#L411-L434](#)
- [accept.rs#L568-L572](#)
- [settlement.rs](#)

#### Description:

RFQ loads `min_notional` into `MarketContext`, but only applies it when deciding whether to place the unfilled remainder on the orderbook. The RFQ quote path validates quantity tick size, price band, price tick size, maker capacity, and maker `min_fill_quantity`, then accepts `filled_qty` without checking `filled_qty * quote.price ≥ market_context.min_notional`.

```
// ...
let filled_qty = state.remaining_qty.min(candidate.quote.quantity);
if let Err(error) = ensure_quantity_matches_min_tick(
    "filled quantity",
    filled_qty,
    context.market_context.min_qty_tick,
) {
    push_quote_error(&mut state.quote_results,
        candidate.quote.maker.clone(), error);
    continue;
}

if let Some(min_fill_quantity) = candidate.quote.min_fill_quantity {
```

```
        if filled_qty < min_fill_quantity {
            push_quote_error(
                &mut state.quote_results,
                candidate.quote.maker.clone(),
                ContractError::InvalidInput("fill below maker
min_fill_quantity".into()),
            );
            continue;
        }
    }

    apply_quote_capacity_allocation(
        candidate.capacity_allocation,
        &mut state.maker_available_balances,
        &mut state.maker_reduce_only_quantities,
    );
    state.seen_quote_payloads.insert(candidate.signed_payload);
    // ...

// ...
if order_quantity * price < market_context.min_notional {
    return Ok(FallbackRoutingOutcome {
        order_quantity: FPDecimal::ZERO,
        dropped_quantity: state.remaining_qty,
    });
}
// ...
```

Injective Core enforces the minimum notional later in the privileged synthetic-trade path. A quote can therefore pass RFQ validation, be selected as a filled quote, and only fail inside Core, reverting the entire AcceptQuote or AcceptSignedIntent transaction instead of recording that quote as rejected and continuing to later quotes or fallback.

### Recommendations:

Before accepting a filled quote, reject or skip it when `filled_qty * candidate.quote.price < market_context.min_notional`.

**Injective:** Resolved with [@0ac8aa41b5...](#)

**Zenith:** Verified.

## [M-2] The evm\_chain\_id should be empty for V1 signatures

SEVERITY: Medium

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: High

### Target

- [intent.sol](#)

### Description:

Takers and makers can use either V1 or V2 signatures. When submitters use Auto mode, the signature version is determined based on evm\_chain\_id. If evm\_chain\_id is non-empty, the signature is treated as V2; otherwise, V1 is selected.

```
SignMode::Auto => {  
  let config = config.expect("config is loaded for auto mode");  
  
  if intent.evm_chain_id.is_some() {  
  
    SignMode::V2  
  } else {  
  
    SignMode::V1  
  }  
}
```

However, there is currently no validation enforcing consistency between V1 signatures and evm\_chain\_id. Users can create a V1 signature with a non-empty evm\_chain\_id, and even 0 is treated as non-empty. As a result, when submitters select Auto mode, the signature is incorrectly interpreted as V2, causing the transaction to revert.

### Recommendations:

For V1 signatures, enforce that evm\_chain\_id must be empty. This validation should be added to both intent.rs and quote\_validation.rs.

**Injective:** Resolved with [@bfca1cdf3d...](#)

**Zenith:** Verified.

## [M-3] Long-lived TP/SL signed intents are not bound to the protected position lifecycle

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

### Target

- [msg.rs#L242-L268](#)
- [intent.rs#L366-L386](#)
- [intent.rs#L391-L435](#)
- [accept.rs#L173-L197](#)
- [state.rs#L250-L258](#)

### Description:

The new update allows MarkPriceGte and MarkPriceLte signed intents to remain valid for roughly 100 years. These intents are bound to (taker, market\_id, subaccount\_nonce, epoch, lane\_version), but not to a concrete position generation, open timestamp, entry price, or position snapshot.

```
pub struct SignedTakerIntent {  
    // ...  
    pub epoch: u64,  
    pub subaccount_nonce: u32,  
    pub lane_version: u64,  
    pub deadline_ms: u64,  
    pub direction: Direction,  
    pub quantity: FPDecimal,  
    pub worst_price: FPDecimal,  
    pub trigger: Trigger,  
    // ...  
}
```

```
// ...  
let max_ttl_ms = if is_tp_sl_trigger(&intent.trigger) {  
    MAX_TP_SL_SIGNED_INTENT_TTL_MS  
} else {  
    MAX_SIGNED_INTENT_TTL_MS  
}
```

```
};  
  
fn is_tp_sl_trigger(trigger: &Trigger) → bool {  
    matches!(trigger, Trigger::MarkPriceGte(_) | Trigger::MarkPriceLte(_))  
}
```

```
// ...  
let reducible_quantity = query_reducible_quantity_for_direction(  
    deps,  
    &args.market_id,  
    taker_subaccount_id,  
    &args.direction,  
)?;  
if args.quantity > reducible_quantity {  
    return Err(ContractError::InvalidInput(  
        "requested taker reduce-only quantity exceeds available reduce-only  
quantity".into(),  
    )  
    .into());  
}  
// ...
```

If the taker closes the original protected position outside `AcceptSignedIntent`, RFQ does not learn that lifecycle event and does not bump `TAKER_LANE_VERSIONS`. The stale TP/SL intent can remain valid. If the taker later opens a new position on the same market and subaccount nonce, the old intent can execute against the new unrelated position as long as the current live reduce-only quantity is sufficient.

### Recommendations:

Bind TP/SL intents to a position generation, open timestamp, or signed position snapshot, and reject execution when the current position no longer matches. Alternatively, cap TP/SL deadlines to a practical renewal period and provide a clear lane invalidation flow when a protected position is closed through RFQ.

**Injective:** Acknowledged.

## 4.2 Low Risk

A total of 1 low risk findings were identified.

[L-1] Updating `allowed_evm_chain_ids` retroactively changes executability of already-signed V2 messages

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

### Target

- [admin.rs#L169-L184](#)
- [helpers.rs#L45-L56](#)
- [intent.rs#L51-L95](#)
- [quote\\_validation.rs#L210-L235](#)

### Description:

The V2 signature digest binds `evm_chain_id`, but execution checks that value against the live `config.allowed_evm_chain_ids`. Admins can update the allowlist after signatures are produced.

```
// ...
if let Some(allowed_evm_chain_ids) = args.allowed_evm_chain_ids {
    config.allowed_evm_chain_ids
    = normalize_allowed_evm_chain_ids(allowed_evm_chain_ids)?;
}
```

```
// ...
let config = if matches!(sign_mode, SignMode::Auto | SignMode::V2) {
    Some(load_config(deps.storage)?)
} else {
    None
};
// ...

let config = config.expect("config is loaded for v2 mode");
```

```
let evm_chain_id = intent
    .evm_chain_id
    .ok_or_else(|| StdError::msg("signed intent evm_chain_id is empty"))?;
ensure_evm_chain_id_allowed(evm_chain_id, &config.allowed_evm_chain_ids)?;
```

```
// ...
if !allowed_evm_chain_ids.contains(&evm_chain_id) {
    return Err(ContractError::InvalidInput(format!(
        "evm_chain_id {} is not allowed",
        evm_chain_id
    )).into());
}
// ...
```

Removing a chain id can brick already-signed V2 quotes or long-lived TP/SL intents that were valid when signed. Adding a chain id can make previously rejected but validly signed messages executable.

### Recommendations:

Include a signed config version or allowlist epoch in V2 signed messages, or document and emit explicit events warning that allowlist changes affect all in-flight V2 signatures. For long-lived TP/SL intents, prefer rejecting execution when the allowlist epoch changed after signing.

**Injective:** Acknowledged.

## 4.3 Informational

A total of 7 informational findings were identified.

[I-1] There is no need to validate the chain ID and contract address in V2 mode

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

### Target

- [intent.sol](#)

### Description:

In V1 mode, `chain_id` and `contract_address` must match the runtime environment values. However, in V2 mode, `evm_chain_id` is needed, and the current contract address is directly entered into the domain separator, so these additional checks are unnecessary.

```
build_signed_taker_intent_digest(deps.api, &env.contract.address, &intent)?;
```

### Recommendations:

Update `validate_signed_intent_shape` function.

```
fn validate_signed_intent_shape(intent: &SignedTakerIntent, sign_mode:
    SignMode) -> StdResult<()> {
    if matches!(sign_mode, SignMode::V1)
        || !intent.chain_id.is_empty()
        || !intent.contract_address.is_empty()
    {
        if intent.chain_id.is_empty() {
            return Err(
                ContractError::InvalidInput("signed intent chain_id is
empty".into()).into(),
            );
        }
    }
}
```

```
    }  
  
    if intent.contract_address.is_empty() {  
        return Err(ContractError::InvalidInput(  
            "signed intent contract_address is empty".into(),  
        )  
        .into());  
    }  
}
```

**Injective:** Acknowledged.

## [I-2] Taker-specific maker quotes dual-signed as V1 and V2 can bypass duplicate detection and fill twice

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

### Target

- [msg.rs#L157-L174](#)
- [quote\\_validation.rs#L189-L240](#)
- [accept.rs#L411-L441](#)

### Description:

Taker-specific maker quotes have `nonce = None`, so they do not consume `MAKER_NONCES`. The update relies on `seen_quote_payloads` to reject duplicate quotes within a single `accept` call, but that dedupe key is the raw serialized `SignQuote` payload.

The new PR adds EIP-712 V2 while keeping legacy V1. For the same economic taker-specific quote, a V1 payload can have `evm_chain_id = None`, while the V2 payload has `evm_chain_id = Some(chain_id)`. Those payloads are semantically the same quote, but their serialized bytes differ, so `seen_quote_payloads` treats them as distinct. If maker infrastructure emits both V1 and V2 signatures as compatibility alternatives during migration, the quote recipient can include both signatures in one direct `AcceptQuote`. In the signed-intent flow, the same overfill can be submitted by the taker-authorized relayer path when the taker's signed intent permits that submission.

The code below shows that `evm_chain_id` changes the serialized dedupe payload, while taker-specific quotes do not consume a maker nonce:

```
// msg.rs
pub struct Quote {
    // ...
    pub signature: Binary,
    // ignore this field if quote for a specific taker
    pub nonce: Option<u64>,
    #[serde(default)]
    pub sign_mode: SignMode,
    #[serde(default, skip_serializing_if = "Option::is_none")]
    pub evm_chain_id: Option<u64>,
    // ...
}
```

```
}
```

```
// quote_validation.rs
// ...
let signed_rfq_id = match binding_mode {
    QuoteBindingMode::Blind { nonce } => {
        validate_maker_nonce(deps, env, &maker, maker_info.subaccount_nonce,
            nonce)?;
        nonce
    }
    QuoteBindingMode::TakerSpecific => input.rfq_id,
};

let sign_quote = SignQuote {
    // ...
    evm_chain_id: quote.evm_chain_id,
    rfq_id: signed_rfq_id,
    // same economic quote fields are included here
    // ...
};

let signed_payload = cosmwasm_std::to_json_binary(&sign_quote)?;

if seen_quote_payloads.contains(signed_payload.as_slice()) {
    return Err(ContractError::InvalidInput("quote
        duplicated".into()).into());
}
```

```
// accept.rs
// ...
state.seen_quote_payloads.insert(candidate.signed_payload);
if let Some(nonce) = candidate.quote.nonce {
    // Blind quotes consume a maker nonce; taker-specific quotes have none.
    // ...
}
```

## Recommendations:

Add a maker quote identity that is independent of signature scheme, preferably a quote id or maker nonce even for taker-specific quotes.

If the intended migration rule is a maker must never emit both formats for the same economic quote, document and enforce that. Contract-side semantic dedupe is still safer because the RFQ contract already attempts to reject duplicate quotes.

**Injective:** Acknowledged.

## [I-3] The mode can not be Auto in the ensure\_allowed\_relayer function

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [intent.sol](#)

### Description:

The mode has already been resolved to either V1 or V2 before calling the ensure\_allowed\_relayer function.

```
pub fn handle_accept_signed_intent(  
    mut deps: DepsMut<InjectiveQueryWrapper>,  
    env: Env,  
    info: MessageInfo,  
    args: AcceptSignedIntentArgs,  
) → StdResult<Response<InjectiveMsgWrapper>> {  
    let verified_sign_mode = match sign_mode {  
        SignMode::Auto ⇒ {  
            if intent.evm_chain_id.is_some() {  
                SignMode::V2  
            } else {  
                SignMode::V1  
            }  
        }  
        SignMode::V1 ⇒ {  
            SignMode::V1  
        }  
        SignMode::V2 ⇒ {  
            SignMode::V2  
        }  
    }  
};  
  
ensure_allowed_relayer(  
    deps.api,  
    &info.sender,  
    intent.allowed_relayer.as_ref(),
```

```
        verified_sign_mode,  
    )?;
```

## Recommendations:

```
fn ensure_allowed_relayer(  
    api: &dyn Api,  
    sender: &Addr,  
    allowed_relayer: Option<&Addr>,  
    sign_mode: SignMode,  
) -> StdResult<()> {  
    let normalized_allowed_relayer = match sign_mode {  
        // In V2, the typed-data schema uses the zero EVM address as the  
        // sentinel for  
        // "no relayer restriction", so runtime authorization must normalize  
        // it the  
        // same way to keep digest semantics and execution semantics aligned.  
        SignMode::V2 => normalize_v2_allowed_relayer(api, allowed_relayer)?,  
        SignMode::V1 | SignMode::Auto => allowed_relayer.cloned(),  
        SignMode::V1 => allowed_relayer.cloned(),  
    };
```

**Injective:** Resolved with [@5d145f139b...](#)

**Zenith:** Verified.

## [I-4] There is no need to call the `validate_signed_intent_shape` function in Auto mode

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

### Target

- [intent.sol](#)

### Description:

When the mode is set to Auto, the `validate_signed_intent_shape` function is called twice.

```
▷ validate_signed_intent_shape(&intent, sign_mode)?;  
  
let verified_sign_mode = match sign_mode {  
  SignMode::Auto => {  
    let config = config.expect("config is loaded for auto mode");  
  
    if intent.evm_chain_id.is_some() {  
▷       validate_signed_intent_shape(&intent, SignMode::V2)?;  
  
        SignMode::V2  
    } else {  
▷       validate_signed_intent_shape(&intent, SignMode::V1)?;  
  
        SignMode::V1  
    }  
  }  
}
```

### Recommendations:

Skip the first function call when the mode is set to Auto.

**Injective:** Acknowledged.

[I-5] V1 `sign_mode` bypasses the EVM-chain allowlist, so the allowlist is not a global signature policy

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

## Target

- [msg.rs#L20-L27](#)
- [msg.rs#L298-L307](#)
- [intent.rs#L51-L97](#)
- [quote\\_validation.rs#L210-L235](#)
- [helpers.rs#L45-L56](#)

## Description:

The PR adds an EVM-chain-id allowlist for EIP-712 V2 signatures, but the allowlist is only checked on the V2 path. When `SignMode::V1` is selected, the contract verifies the legacy payload and does not consult `allowed_evm_chain_ids`. Auto mode also falls back to V1 when `evm_chain_id` is absent.

```
// ...
pub struct AcceptSignedIntentArgs {
    pub intent: SignedTakerIntent,
    pub taker_signature: Binary,
    pub quotes: Vec<Quote>,
    pub quote_rfq_id: Option<u64>,
    #[serde(default)]
    pub sign_mode: SignMode,
}
```

```
// ...
let config = if matches!(sign_mode, SignMode::Auto | SignMode::V2) {
    Some(load_config(deps.storage)?)
} else {
    None
};
```

```
// ...
match sign_mode {
  // ...
  SignMode::V1 => {
    let _msg_hash = verify_signature(deps.api, &payload,
    &taker_signature, &intent.taker)?;
    SignMode::V1
  }
  SignMode::V2 => {
    let config = config.expect("config is loaded for v2 mode");
    let evm_chain_id = intent
      .evm_chain_id
      .ok_or_else(|| StdError::msg("signed intent evm_chain_id is
empty"))?;
    ensure_evm_chain_id_allowed(evm_chain_id,
    &config.allowed_evm_chain_ids)?;
    let digest =
      build_signed_taker_intent_digest(deps.api,
    &env.contract.address, &intent)?;
    let _msg_hash =
      verify_eip712_signature(deps.api, &digest, &taker_signature,
    &intent.taker)?;
    SignMode::V2
  }
  // Auto falls back to V1 when evm_chain_id is absent.
  // ...
}
```

The new allowlist is V2-only and therefore not a global signature rule while valid legacy V1 signatures remain executable.

### Recommendations:

If the allowlist is intended as a global control, add a config switch to disable V1 and reject V1 once migration is complete. If V1 support is intentionally permanent, document that `allowed_evm_chain_ids` applies to V2 only.

**Injective:** Acknowledged.

## [I-6] Long TP/SL lifetime is granted solely by trigger type, allowing always-satisfied long-lived immediate intents

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

### Target

- [msg.rs#L35-L40](#)
- [intent.rs#L251-L335](#)
- [intent.rs#L375-L392](#)
- [intent.rs#L419-L435](#)

### Description:

The new update selects the 100-year TP/SL lifetime only by checking whether the trigger type is `MarkPriceGte` or `MarkPriceLte`. The code does not validate that the trigger is a meaningful TP/SL condition.

```
// ...
pub enum Trigger {
    Immediate,
    MarkPriceGte(FPDecimal),
    MarkPriceLte(FPDecimal),
}
```

```
// ...
let max_ttl_ms = if is_tp_sl_trigger(&intent.trigger) {
    MAX_TP_SL_SIGNED_INTENT_TTL_MS
} else {
    MAX_SIGNED_INTENT_TTL_MS
};

fn is_tp_sl_trigger(trigger: &Trigger) → bool {
    matches!(trigger, Trigger::MarkPriceGte(_) | Trigger::MarkPriceLte(_))
}
```

```
// ...
fn ensure_trigger_satisfied(trigger: &Trigger, mark_price: FPDecimal) →
    StdResult<()> {
    let triggered = match trigger {
        Trigger::Immediate ⇒ true,
        Trigger::MarkPriceGte(price) ⇒ mark_price ≥ *price,
        Trigger::MarkPriceLte(price) ⇒ mark_price ≤ *price,
    };
    // MarkPriceGte(0) is satisfied for any non-negative mark price.
    // ...
}
```

An intent can therefore use an always-satisfied trigger, such as `MarkPriceGte(0)`, and receive the long TP/SL lifetime while behaving like an immediate intent whenever quotes are available.

### Recommendations:

Do not use trigger type alone as the gate for long lifetimes. Add a clear signed intent kind, reject always-satisfied trigger thresholds, and require the trigger direction to match the protected position direction.

**Injective:** Acknowledged.

## [I-7] Signed taker\_nonce\_time\_window\_ms can remove the practical RFQ-id freshness bound

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [msg.rs#L177-L183](#)
- [msg.rs#L252-L256](#)
- [accept.rs#L241-L249](#)
- [state.rs#L261-L274](#)

### Description:

The update makes taker\_nonce\_time\_window\_ms a signer-supplied field on AcceptQuoteArgs and SignedTakerIntent. add\_nonce uses that value directly to decide whether the RFQ id is close enough to the execution block timestamp. There is no contract-side maximum or minimum clamp.

```
// ...
pub struct AcceptQuoteArgs {
    pub rfq_id: u64,
    pub taker_nonce_time_window_ms: u64,
    // ...
}
```

```
// ...
pub struct SignedTakerIntent {
    // ...
    pub rfq_id: u64,
    pub taker_nonce_time_window_ms: u64,
    // ...
}
```

```
// ...
add_nonce(
    deps.storage,
```

```
block_time_ms(env),  
taker,  
args.rfq_id,  
args.taker_nonce_time_window_ms,  
)?;
```

```
// ...  
let min_allowed =  
  block_time_millis.saturating_sub(taker_nonce_time_window_ms);  
let max_allowed =  
  block_time_millis.saturating_add(taker_nonce_time_window_ms);  
if nonce < min_allowed || nonce > max_allowed {  
  return Err(StdError::msg("nonce outside window"));  
}  
// ...
```

An off-chain signing flow can cause the taker to sign an intent with an overly large `taker_nonce_time_window_ms`, weakening the intended RFQ-id freshness bound for that still unexecuted authorization.

### Recommendations:

Enforce a contract-side maximum and minimum for `taker_nonce_time_window_ms`.

**Injective:** Resolved with [@1c2bba5b67 ...](#)

**Zenith:** Verified.